RFC — Motor antifraude Pix com regras pontuadas, histórico local e decisão idempotente

TIME	Hugo Gomes · Leandro Apolinário Leite · Mateus Carósio
DATA	17/10/2026
VERSÃO	2

Contextualização

Entendendo o problema

O sistema chamador envia cada pedido de Pix e o motor responde, antes do dinheiro sair, se ele é `approved`, `two_factor_required` ou `rejected`. A resposta precisa vir em poucos segundos, sem analista e sem consulta externa pesada, e sempre com um motivo explicável. Uma transação isolada diz pouco: o motor compara o Pix com o que o pagador, o destinatário e o dispositivo já fizeram.

Dois erros custam caro. Aprovar uma fraude libera um dinheiro difícil de recuperar; barrar um cliente bom (o aluguel pago pela primeira vez a um novo locador) gera fricção e prejuízo. Há ainda um erro de consistência: o mesmo Pix receber duas decisões diferentes, ou uma rajada paralela passar sem ser vista porque cada requisição leu um histórico incompleto. Cada Pix recebido e sua análise ficam gravados e consultáveis.

Fora do escopo: onboarding e cadastro de contas, front-end, outras APIs, movimentar dinheiro, validar o segundo fator (o motor só o solicita), regra de saldo (o payload não traz saldo) e modelos de machine learning.

Explicando a solução de forma macro

A API valida e normaliza o Pix, trava as chaves envolvidas (o próprio `id`, o pagador e a conta destino) dentro de uma transação PostgreSQL, lê o histórico local já gravado, avalia 12 regras determinísticas e soma pontos. Cada regra é uma função pura (transação + contexto de histórico → pontos e detalhe), o que permite testá-la sem banco; as camadas seguem o Base Service (rota, serviço, repositório). A soma vira o status por faixas, com um teto para os sinais de valor, de modo que valor alto sozinho nunca reprove. Transação, análise e regras acionadas são gravadas na mesma transação de banco. O `id` do Pix é a chave de idempotência, e o `payload_hash` distingue reenvio legítimo de reuso indevido do `id`.

Alternativas consideradas e descartadas:

- **Primeira regra que casa decide (sem somar)** — descartada porque perde contexto: três sinais moderados juntos valem mais que cada um sozinho. Ganharia se o cliente exigisse bloqueios categóricos, como lista de contas proibidas.
- **Reprovar qualquer alerta** — descartada porque dispara falsos positivos e trava quem só quer pagar. Ganharia se a perda média por fraude fosse muito maior que o custo de fricção, como em contas já marcadas como de alto risco.
- **Consultar bases externas a cada Pix** — descartada porque a latência e a dependência externa quebram o "poucos segundos". Ganharia se houvesse uma base de reputação com SLA em milissegundos ou cache local confiável.
- **Modelo de machine learning** — descartado porque exige dados rotulados de fraude confirmada, avaliação e monitoramento que não existem aqui. Ganharia com esse histórico rotulado; as regras gravadas por Pix servem de base auditável para isso.

Implementação

Rotas

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`POST`	`/pix/transaction`	Valida, analisa, grava e devolve a decisão. **Idempotente:** `id` do Pix com `UNIQUE` no banco + `payload_hash` do payload normalizado.	`id`; `amount` (centavos); `transaction_date` (com fuso); `source_account.owner.document_number` e `opening_date`; `destination_account` (`participant`, `branch`, `account_number`, `owner.document_number`, `opening_date`); `source.device_id`, `channel`, `ip`, `geolocation` (opcional)	**201** análise criada, com `key`, `status` e `metadata` (`risk_score`, `reasons`, `rules_version`). **200** mesmo `id` e mesmo payload: devolve a análise já gravada, sem reprocessar. **400** corpo ilegível ou JSON malformado. **409** mesmo `id` com payload diferente. **422** campo ausente ou de tipo errado, `amount` não inteiro ou ≤ 0, data sem fuso, documento sem 11 ou 14 dígitos, latitude ou longitude fora da faixa. **500** falha interna: tudo é revertido e o reenvio é seguro.

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS E QUANDO)
`GET`	`/pix/transaction/{id}`	Devolve a análise gravada (somente leitura). `{id}` é o `id` do Pix, não o `key`.	`id` na URL	**200** `key`, `status` e `metadata`. **404** nenhuma análise para esse `id`.

Banco de Dados (Somente diagrama)

PIX_TRANSACTION

`string`	`id`	PK	*id do Pix, usado no GET*
`string`	`payload_hash`		*SHA-256 do payload normalizado; detecta reuso do id com payload diferente*
`bigint`	`amount`		*valor em centavos*
`timestamptz`	`transaction_date`		*em UTC; âncora de todas as janelas*
`string`	`source_document`		*CPF/CNPJ só com dígitos; chave do pagador; índice com transaction_date*
`string`	`source_branch`		
`string`	`source_account_number`		
`timestamptz`	`source_opening_date`		
`string`	`destination_participant`		*ISPB*
`string`	`destination_branch`		
`string`	`destination_account_number`		
`string`	`destination_account_key`		*participant + agência + conta; coluna gerada; índice com transaction_date*
`string`	`destination_document`		*CPF/CNPJ só com dígitos*
`timestamptz`	`destination_opening_date`		
`string`	`device_id`		*índice com transaction_date*
`string`	`channel`		
`string`	`ip`		
`float`	`latitude`		*opcional*
`float`	`longitude`		*opcional*
`jsonb`	`raw_payload`		*payload original, para auditoria*
`timestamptz`	`created_at`		*quando a API recebeu*

1 — *recebe uma análise* — **1**

FRAUD_ANALYSIS

`uuid`	`key`	PK	*campo key da resposta*
`string`	`transaction_id`	FK UK	*uma análise por Pix*
`string`	`status`		*approved, rejected ou two_factor_required*
`int`	`risk_score`		*soma final dos pontos*
`string`	`rules_version`		*versão da política aplicada*
`timestamptz`	`analyzed_at`		*quando a decisão foi tomada*

1 — *registra regras acionadas* — **0..N**

RULE_HIT

`bigint`	`id`	PK	
`uuid`	`analysis_key`	FK	*da análise*
`string`	`rule_code`		*R1 a R12*
`smallint`	`points`		*pontos antes do teto*
`string`	`detail`		*valores observados*

Fluxos

POST /pix/transaction — caminho feliz

1. Validar a requisição (`400`/`422`) e normalizar: documentos só com dígitos, datas em UTC, geolocalização opcional. Calcular o `payload_hash` sobre o payload normalizado.
2. Abrir a transação PostgreSQL e obter `pg_advisory_xact_lock` sobre três chaves, sempre na mesma ordem (`id`, pagador, conta destino) para evitar deadlock. Os locks caem no commit.
3. Buscar o `id`. Se existir com o mesmo hash, devolver `200` com a análise existente. Se o hash for diferente, devolver `409`. Nos dois casos, nada é reprocessado.
4. Carregar o contexto de histórico com consultas agregadas e indexadas. As janelas são relativas à `transaction_date` do próprio Pix, usam só registros com data menor ou igual a ela e excluem o próprio `id`.
5. Executar R1 a R12 em ordem fixa, aplicar modificadores e teto, somar e mapear a faixa em `status`.
6. Gravar `PIX_TRANSACTION`, `FRAUD_ANALYSIS` e `RULE_HIT` na mesma transação, fazer commit e devolver `201` com `key`, `status` e `metadata`.

Regras avaliadas no passo 5 (parâmetros iniciais, calibrados com os cenários de teste e repetidos no README)

REGRA	LÓGICA E PARÂMETROS	PTS	POR QUÊ
R1 Limite	`amount` ≥ R$ 5.000	**2**	Valor alto aumenta a exposição, mas não prova fraude: sozinho leva só a 2FA. O piso é um ponto de partida para Pix de uso comum alto (aluguel, parcela).
R2 Velocity	≥ 5 Pix do pagador em 60 s, contando o atual, qualquer status	**2**	Pessoa raramente faz 5 Pix por minuto; script faz. Conta tentativas rejeitadas, porque a rajada é o sinal.
R3 Montante acumulado	Soma do pagador em 10 min ≥ R$ 10.000, com pelo menos 2 Pix na janela	**2**	Pega valor fatiado que escapa do R1. Exigir 2 Pix evita contar duas vezes um único Pix grande.

REGRA	LÓGICA E PARÂMETROS	PTS	POR QUÊ
R4 Desvio de perfil	`amount` ≥ 5× a mediana dos últimos 20 Pix não rejeitados do pagador, com histórico ≥ 5 e `amount` ≥ R$ 500	**2**	A mediana resiste a valores atípicos; 5× evita oscilação normal; o piso evita disparar em valores pequenos; o histórico mínimo evita inventar perfil de cliente novo.
R5 Contraparte nova	Pagador sem Pix anterior para essa conta destino, com histórico ≥ 3 e `amount` ≥ R$ 1.000	**1**	Destinatário novo é contexto, não motivo isolado. O histórico mínimo impede que o primeiro Pix de todo cliente pontue.
R6 Destino recém-aberto	Conta destino com ≤ 30 dias na `transaction_date`, medidos em tempo decorrido, e `amount` ≥ R$ 1.000	**2**	Contas recentes concentram risco de conta de passagem. 30 dias evita a fronteira apertada de 7 dias.
R7 Dispositivo novo	`device_id` fora dos dispositivos dos últimos 20 Pix não rejeitados do pagador, com histórico ≥ 3 e `amount` ≥ R$ 500	**2**	Troca de aparelho é legítima, por isso só pontua com valor relevante e perfil existente.
R8 Fracionamento	≥ 3 Pix do pagador para a mesma conta destino em 5 min, cada um < R$ 1.000, com soma ≥ R$ 2.000	**2**	Detecta quem fica abaixo dos pisos de R5 e R6 dividindo o valor.
R9 Destino concentrador	Conta destino recebeu de ≥ 5 pagadores distintos em 60 min, com soma ≥ R$ 10.000, contando o atual; não vale para destinatário CNPJ com conta > 30 dias	**3**	Padrão de conta de passagem. A exceção protege comerciantes legítimos.
R10 Pagador recente	Conta pagadora com ≤ 30 dias e `amount` ≥ R$ 2.000	**1**	Sinal complementar de idade da conta.
R11 Deslocamento impossível	Com geolocalização nos dois Pix, o Pix anterior do pagador foi há ≤ 60 min, a distância é ≥ 50 km e a velocidade implícita é > 900 km/h	**3**	Sugere sessão usada de outro lugar. 900 km/h é a ordem de um voo comercial; 50 km filtra ruído de GPS. Sem geolocalização, não pontua.
R12 Dispositivo compartilhado	Mesmo `device_id` usado por ≥ 3 documentos pagadores distintos em 24 h	**2**	Um aparelho servindo várias titularidades sugere fazenda de contas.

Combinação. O teto de **3 pontos** vale para a soma de R1 + R3 + R4 (família "valor"), e assim os três não contam a mesma coisa várias vezes. Se `source_document` = `destination_document`, R5, R6 e R9 não pontuam. O total define o status: **0–1** `approved`; **2–4** `two_factor_required`; **≥ 5** `rejected`. Não há exceção nem desvio de política: a ordem de execução é fixa apenas para tornar a lista de motivos estável, e não há short-circuit. Como valor alto soma no máximo 3, reprovar exige pelo menos 2 pontos de outros sinais. Na prática, o cliente com mediana de R$ 200 que paga R$ 6.000 pela primeira vez a um locador, no aparelho de sempre, soma 3 + 1 = 4 e recebe 2FA.

O histórico de confiança (R4, R5, R7) exclui apenas Pix `rejected`. O motor não recebe o resultado do 2FA, e contar só os `approved` faria o cliente do aluguel cair em 2FA todo mês. O histórico de rajada (R2, R3, R8, R9, R11, R12) conta qualquer status.

POST /pix/transaction — falha: mesmo id em duas requisições simultâneas

1. As duas requisições disputam o lock do `id`; a segunda espera.
2. Quando a primeira faz commit, a segunda segue, encontra o `id` gravado e compara o hash: mesmo hash devolve `200` com a decisão gravada; hash diferente devolve `409`.
3. Se um bug furar o lock, a restrição `UNIQUE` de `id` e `transaction_id` rejeita o segundo `INSERT`. A requisição faz rollback, relê o registro e trata como reenvio. Nunca existem duas análises.

POST /pix/transaction — falha: rajada paralela do mesmo pagador

1. Cinco Pix diferentes do mesmo pagador chegam juntos. Sem lock, cada um leria um histórico vazio e o R2 nunca dispararia.
2. O lock por pagador faz as requisições passarem uma de cada vez. Cada uma enxerga o que as anteriores já gravaram, e a quinta vê 4 anteriores mais o atual: o R2 dispara.
3. A trava também cobre a conta destino, para o R9. O R12 (janela de 24 h) fica sem trava: uma corrida de milissegundos quase não muda esse sinal.

GET /pix/transaction/{id} — caminho feliz e falha

1. Buscar a análise pelo `id` do Pix e devolver `key`, `status` e `metadata` (`200`).
2. Se não existir, devolver `404`, sem criar registro nem executar regras.

Principal desafio

- **Qual é:** decidir de forma determinística a partir de um histórico que outras requisições estão alterando ao mesmo tempo, sem punir o cliente legítimo.
- **Por que é difícil:** velocity, montante e contraparte dependem de ler o que acabou de ser gravado, e a rajada paralela, justamente o ataque que o R2 quer pegar, faz cada requisição ver um histórico vazio. Do outro lado, sinais correlacionados somados jogam em `rejected` quem só paga o aluguel.
- **Como o desenho resolve:** `pg_advisory_xact_lock` sobre `id`, pagador e conta destino na mesma transação que lê o histórico e grava a análise; `UNIQUE` como última barreira; teto de 3 pontos na família valor, históricos mínimos antes de pontuar perfil e rejeição só a partir de 5 pontos.